

Lecture 6: Data Acquisition - Scraping Dynamic Websites

James Sears*

AFRE 891/991 FS 25

Michigan State University

*Parts of these slides are adapted from [“Data Science for Economists”](#) by Grant McDermott.

Table of Contents

1. Prologue
2. Scraping Dynamic Websites

Prologue

Packages for Today

Today we'll pick up with scraping dynamic websites.

For this we'll need the following packages:

Static:

```
pacman::p_load(lubridate, rvest, stringr, tidyverse, tidyr)
```

and for dynamic scraping:

```
pacman::p_load(chromote, selenider)
```

Scraping Dynamic Websites

Dynamic Websites

Dynamic client-side websites are those are rendered dynamically and require **interaction** in order to obtain desired information.

While **rvest** can help us retrieve urls and element contents, it can't interact with **dynamically-generated or interactive** websites.¹

To do that, we're going to take advantage of **browser automation** methods.

¹ **rvest** has some functionality for live site interaction, but it is not as complete as the other options we'll see shortly.

Browser Automation

The main idea behind browser automation is this:

If a website requires pointing/clicking/typing to reveal the contents we want, why not have software "drive" a browser for us?

Old Approach: RSelenium

- R bindings for the Selenium webdriver
- Requires some additional software (i.e. Docker)
- Additional steps to get it working with Mac OS X

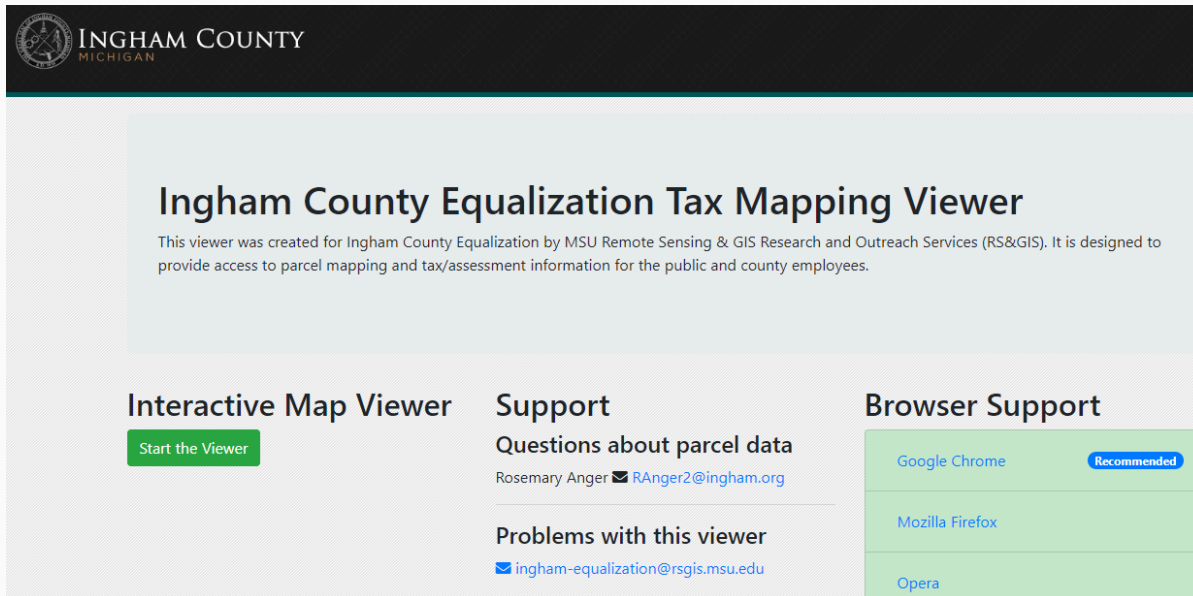
New Approach: selenider

- Uses **chromote** or Selenium automation tools to drive a Chrome browser
- Syntax more closely matches tidyverse
- Can't do some more complex things yet (i.e. click-and-hold, click a point)

Application: EL Properties

Let's work through an example of an interactive workflow for **property characteristics in East Lansing**.

Suppose you want to obtain property information for a known address within East Lansing. The city has a handy **interactive map viewer** for tax assessment info.



The screenshot shows the homepage of the Ingham County Equalization Tax Mapping Viewer. At the top is the Ingham County Michigan logo. The main heading is "Ingham County Equalization Tax Mapping Viewer". Below this is a descriptive paragraph: "This viewer was created for Ingham County Equalization by MSU Remote Sensing & GIS Research and Outreach Services (RS&GIS). It is designed to provide access to parcel mapping and tax/assessment information for the public and county employees." The page is divided into three columns. The left column, titled "Interactive Map Viewer", contains a green button labeled "Start the Viewer". The middle column, titled "Support", includes the text "Questions about parcel data" followed by "Rosemary Anger" and an email link "RAnger2@ingham.org". Below this is "Problems with this viewer" followed by an email link "ingham-equalization@rsgis.msu.edu". The right column, titled "Browser Support", lists three browser options: "Google Chrome" (marked as "Recommended"), "Mozilla Firefox", and "Opera".

INGHAM COUNTY
MICHIGAN

Ingham County Equalization Tax Mapping Viewer

This viewer was created for Ingham County Equalization by MSU Remote Sensing & GIS Research and Outreach Services (RS&GIS). It is designed to provide access to parcel mapping and tax/assessment information for the public and county employees.

Interactive Map Viewer

[Start the Viewer](#)

Support

Questions about parcel data
Rosemary Anger ✉ RAnger2@ingham.org

Problems with this viewer
✉ ingham-equalization@rsgis.msu.edu

Browser Support

Google Chrome	Recommended
Mozilla Firefox	
Opera	

Application: EL Properties

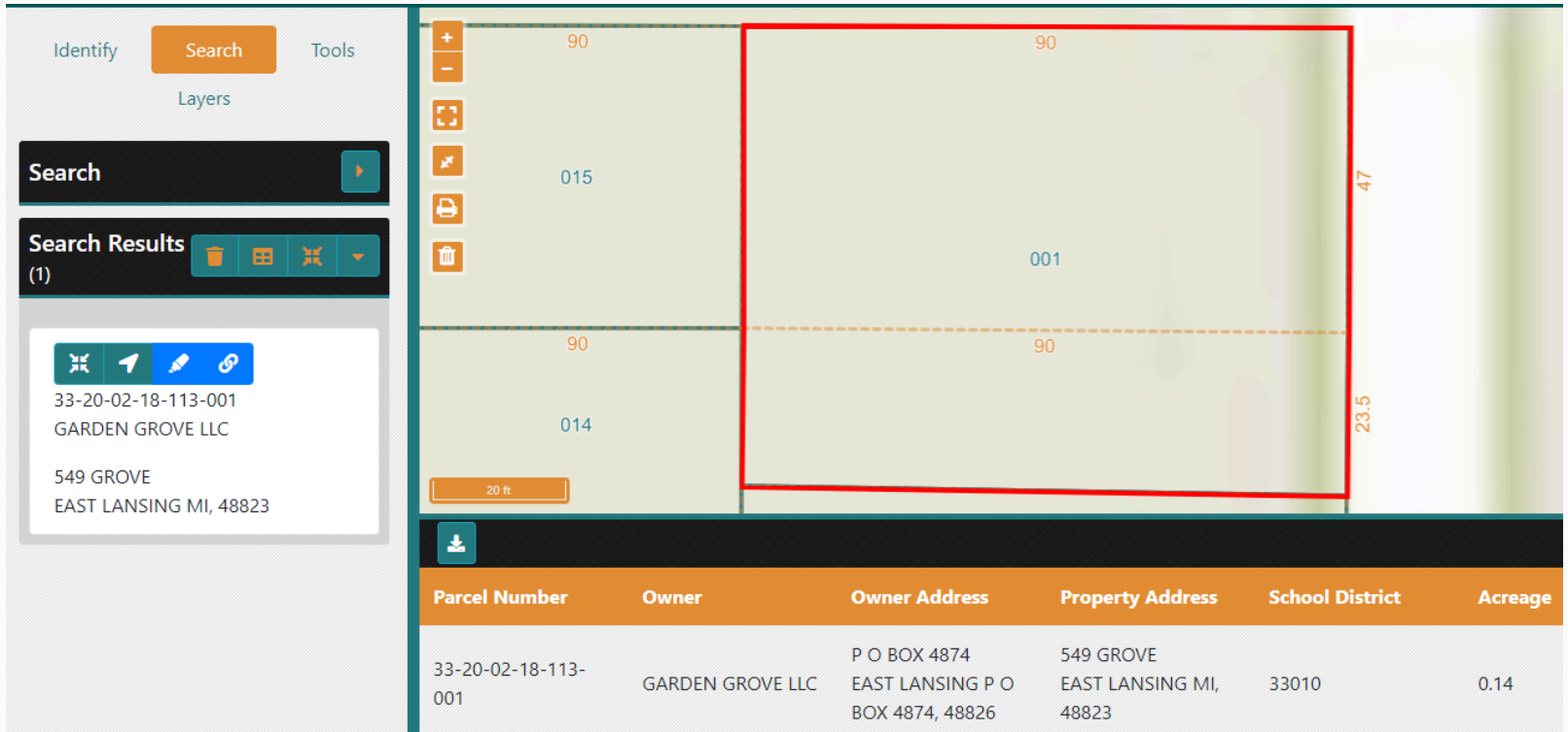
Suppose we wanted to get information about the property at 549 Grove St. from the parcel map.

Doing this takes a couple steps:

1. Launch the [Map Viewer](#)
2. Click on "Search"
3. Type address components (number and street name) into the search bar, hit Go
4. Click on "Snap to Result" button to zoom in on the parcel
5. Click on "Toggle Table" to show information

Application: EL Properties

Unfortunately, this doesn't have the property details we are interested in.



The screenshot displays a web application interface for property information. On the left, a sidebar contains a search bar and a search results panel. The search results panel shows one result for parcel 33-20-02-18-113-001, owned by GARDEN GROVE LLC, located at 549 GROVE, EAST LANSING MI, 48823. The main area features a map with a red bounding box highlighting a specific property. The map includes labels for parcel numbers (90, 015, 001, 90, 014) and dimensions (20 ft, 47, 23.5). Below the map is a table with the following data:

Parcel Number	Owner	Owner Address	Property Address	School District	Acreage
33-20-02-18-113-001	GARDEN GROVE LLC	P O BOX 4874 EAST LANSING P O BOX 4874, 48826	549 GROVE EAST LANSING MI, 48823	33010	0.14

Application: EL Properties

Fortunately, there is a link to more information on a different site:



The image shows a screenshot of a property card. At the top, there is a horizontal bar with four icons: a white cross on a teal background, a white arrow on a teal background, a white pencil on a blue background, and a white link icon on a blue background. Below this bar, the text "33-20-02-18-113-001" is displayed in a large, bold, black font. Underneath that, "GARDEN GROVE LLC" is written in a slightly smaller, bold, black font. Further down, "549 GROVE" is shown in a bold, black font, followed by "EAST LANSING MI, 48823" in a bold, black font. To the right of the address, there is a white rectangular box with a black border containing the text "View on bsaonline [SECURE]" in a black font. The entire card is set against a light gray background with a subtle pattern.

33-20-02-18-113-001

GARDEN GROVE LLC

549 GROVE

EAST LANSING MI, 48823

View on bsaonline [SECURE]

Application: EL Properties

Which *unfortunately* requires verification

[BS&A Online](#) [Home](#) [Services](#) [Municipalities](#) [Currently not signed in](#)

Ingham County | MI

powered by **BS&A** SOFTWARE

Search: [All Records](#) By: [Address](#) [Search](#)

☐ Use Advanced Address Search

SERVICES

[Public Records Search](#)

[All Record Search](#)

[Assessing Search](#)

[Current Tax Search](#)

[Delinquent Tax Search](#)

PERFORMANCE DATA

[MI Community Financial Dashboard](#)

[Performance Dashboard](#)

MUNICIPALITIES

[Select a Municipality](#)

ACCOUNT

[Sign In](#)

[Register](#)

[Why Register?](#)

**Security Verification**
For security purposes, click the checkbox below.

Verify Code

Check the box below to verify you are a human. This security feature helps prevent automated programs from accessing unauthorized information.

☐ I'm not a robot 

[Privacy](#) [Terms](#)

****Disclaimer:** BS&A Software provides BS&A Online as a way for municipalities to display information online and is not responsible for the content or accuracy of the data herein. This data is provided for reference only and WITHOUT WARRANTY of any kind, expressed or inferred. Please contact your local municipality if you believe there are errors in the data.

Application: EL Properties

Which *fortunately* has the characteristics we want once we get past this!

☐ Building Information - 1676 sq ft 1 3/4 STORY (Residential)

[Back To Top](#)

General

Floor Area	1,676 sq ft
Garage Area	216 sq ft
Foundation Size	1,024 sq ft
Year Built	1922
Occupancy	Single Family
Effective Age	52 yrs
Percent Complete	0%
AC w/Separate Ducts	No
Basement Rooms	3
1st Floor Rooms	7
2nd Floor Rooms	2
Bedrooms	7

Estimated TCV	\$193,781
Basement Area	1,006 sq ft
Year Remodeled	No Data to Display
Class	C
Tri-Level	No
Heat	Forced Air w/ Ducts
Wood Stove Add-on	No
Water	Public Water
Sewer	Public Sewer
Style	1 3/4 STORY

Area Detail - Basic Building Areas

Height	Foundation	Exterior	Area	Heated
1.75 Story	Basement	Siding	870 sq ft	1.75 Story
1 Story	Basement	Brick	136 sq ft	1 Story
1 Story	Crawl Space	Siding	18 sq ft	1 Story

Workflow

Let's now work through these steps in the context of an automated browser workflow:

1. Launch an automated browser session
2. Navigate to the Map Viewer
3. Search by street name + house number
4. Retrieve displayed info
5. Click detail link
6. Pass verification test
7. Scrape desired info

1. Launch Automated Session

First, we need to **initiate an automated browser session** with

```
selenider_session().2
```

```
session ← selenider_session(  
  browser = "chrome", # browser to drive  
  session = "chromote", # set backend to chromote  
  timeout = 10 # set timeout to 10 sec  
)
```

We've now created a **local session**

- Accessed anywhere in a given script/RMarkdown file
- Closes automatically when the script/file finishes
- If started within a function, closes when the function is done running

² The full list of **selenider** functions can be found [here](#)

View the Session

If you want to **view the browser session**, we need to add the argument

```
options = chromote_options(headless = F)
```

Which will open the session in your Chrome browser

```
session ← selenider_session(  
  browser = "chrome", # browser to drive  
  options = chromote_options(headless = F), # view the session  
  "chromote", # set backend to chromote  
  timeout = 10 # set timeout to 10 sec  
)
```

2. Navigate to Map Viewer

Let's start by navigating to the main [Tax Mapping Viewer page](#) and launch the viewer

- `open_url("URL")`

```
open_url("https://ingham-equalization.rsgis.msu.edu/")
```

2. Navigate to Map Viewer

Check that we ended up at the desired url:

```
current_url()
```

```
## [1] "https://ingham-equalization.rsgis.msu.edu/"
```

Global Functions

Other actions that apply globally to the entire page:

Function	Task
<code>open_url()</code>	navigate to the indicated url
<code>current_url()</code>	get the current url
<code>get_page_source()</code>	get the page's HTML
<code>back()</code> , <code>forward()</code>	navigate forward or back
<code>reload()</code> , <code>refresh()</code>	reload the current page
<code>take_screenshot()</code>	take screenshot of current image
<code>execute_js_fn()</code> , <code>execute_js_expr()</code>	execute a JavaScript function

2. Navigate to Map Viewer

Now we'll **click the "Start the Viewer" button** to launch the app.

Involved Steps:

- **Select** the button element by its CSS selector (`find_element()`)
- **Scroll to** the button (`elem_scroll_to()`)
- **Click** the button (`elem_click()`)

Selection Functions

Some additional selection functions:

Function	Task
<code>s()</code> , <code>ss()</code>	select HTML elements (without specifying the session)
<code>find_element()</code>	find a single HTML child element (yields a <code>selenium-element</code> object)
<code>find_elements()</code>	find multiple HTML child elements (yields a <code>selenium-elements</code> object)
<code>elem_filter()</code> , <code>elem_find()</code>	extract a subset of HTML elements

Selection Functions

Hold up James, you're telling me that there are **six** different functions ??
Which should I be using?

Thoughts:

- `s()` is equivalent to `session %>% find_element()`
- `ss()` is equivalent to `session %>% find_elements()`
- `s()/find_element()` and `ss()/find_elements()` let you search by
 - CSS Selector (`css`)
 - XPath (`xpath`)
 - element ID (`id`), class (`class_name`), name (`name`)
- `elem_find()` and `elem_filter()` let us search by **conditions**
 - i.e. has certain text, has an attribute with a certain value, is visible, has a certain number of elements
 - We'll see helper functions that will help us out with this in a bit

Element Properties

And functions for obtaining element properties

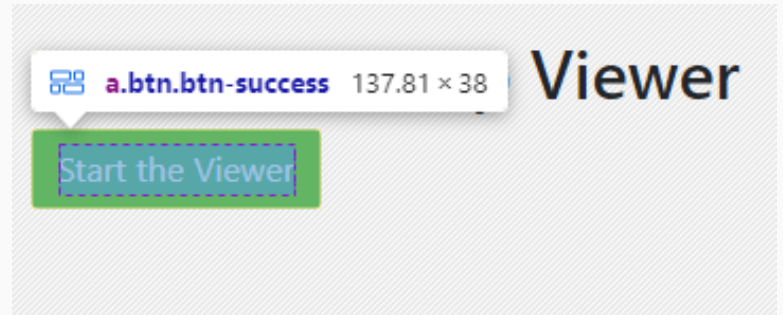
Function	Task
<code>elem_attr()</code> , <code>elem_attrs()</code> , <code>elem_value()</code>	Get attributes of an element
<code>elem_css_property()</code>	Get a CSS property of an element
<code>elem_name()</code>	Get the tag name of an element
<code>elem_size()</code> , <code>length(seleniumider- elements)</code>	get the number of elements in a collection
<code>elem_text()</code>	Get the text inside an element
<code>elem_equal()</code>	Test if two elements are equal

Actions

Function	Task
<code>elem_click()</code> , <code>elem_double_click()</code> , <code>elem_right_click()</code>	Click on an element
<code>elem_hover()</code> , <code>elem_focus</code>	Hover over an element
<code>elem_scroll_to()</code>	Scroll to an element
<code>elem_select()</code>	Select an HTML element
<code>elem_set_value()</code> , <code>elem__send_keys()</code> , <code>elem_clear_value()</code>	Set the value of an input
<code>elem_submit()</code>	Submit an element
<code>keys</code>	list of special keys

2. Navigate to Map Viewer

```
session %>%  
  find_element("a.btn.btn-  
success") %>% # select the button  
by CSS Selector  
  elem_text()
```



```
## [1] "Start the Viewer"
```

```
session %>%  
  find_element("a.btn.btn-success") %>%  
  elem_scroll_to() %>% # move the cursor to the button  
  elem_click() # click the button  
  current_url()
```

```
## [1] "https://ingham-equalization.rsgis.msu.edu/Viewer"
```

3. Search by Address

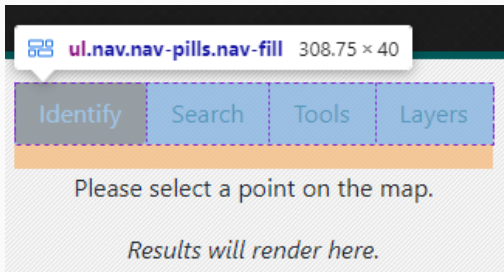
We next want to select + click on the "Search" tab button.

To do this, we'll take advantage of **relative position** selector functions.

Function	Task
<code>elem_parent()</code>	select the element that contains the current element
<code>elem_ancestors()</code>	select every element containing the current element (children, grandchildren, etc.
<code>elem_siblings()</code>	select every element which has the same parent as the current element (e.g. list)
<code>elem_children()</code>	select every element connected to and directly below the current element
<code>elem_descendants()</code>	select every element that is contained by the current element (any type of ancestor)

3. Search by Address

Select + click on the "Search" tab button.

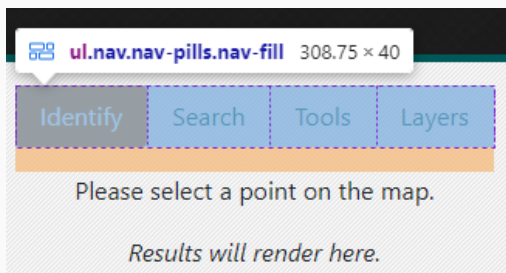


```
session %>%  
  find_element("nav.info-panel") %>% # get  
  full_nav_panel  
  elem_children()
```

```
## { selenider_elements (3) }  
## [1] <ul role="navigation" class="nav nav-  
pills nav-fill"><li class="nav-item"><a ...  
## [2] <div class=""><div  
class="hide_container"><button class="btn btn-  
info">Select ...  
## [3] <div class="hide_container" id="layers-  
tab"><div class="catalog searchable">< ...
```

3. Search by Address

Select + click on the "Search" tab button.



```
session %>%  
  find_element("ul.nav.nav-pills.nav-fill")  
%>% # just the button menu  
  elem_children()
```

```
## { selenider_elements (4) }  
## [1] <li class="nav-item"><a href="#"  
class="nav-link active">Identify</a></li>  
## [2] <li class="nav-item"><a href="#"  
class="nav-link">Search</a></li>  
## [3] <li class="mobile--hidden nav-item"><a  
href="#" class="nav-link">Tools</a></li>  
## [4] <li class="nav-item"><a href="#"  
class="nav-link">Layers</a></li>
```

3. Search by Address

We can also use **condition functions** to help with selection.

Function	Task
<code>has_text(), has_exact_text()</code>	has text that partially or fully matches
<code>has_attr(), attr_contains(), has_value()</code>	does the attribute match a value
<code>has_css_property()</code>	does an element's CSS property match a value (class, id, title, etc.)
<code>has_name()</code>	does an element have a tag name
<code>has_length(), has_size(), has_at_least()</code>	does a collection have a certain number of elements
<code>is_present(), is_in_dom(), is_absent()</code>	does an element exist
<code>is_visible(), is_displayed(), is_hidden(), is_invisible()</code>	is an element visible

3. Search by Address

We can also use **condition functions** to help with selection.

- `find_elements()` to yield a `selenium_element`
- `elem_find()` to extract a specific subset of HTML elements
 - `has_text()` to grab the element with the text "Search"
- `elem_click()` to click the selected button.

```
session %>%  
  find_element("ul.nav.nav-pills.nav-fill") %>%  
  find_elements("li.nav-item") %>%  
  elem_find(has_text("Search")) %>%  
  elem_click()
```

3. Search by Address

Checking to make sure we've activated the "Search" menu tab:

```
session %>%  
  find_element("ul.nav.nav-pills.nav-fill") %>%  
  elem_children()  
  
## { selenider_elements (4) }  
## [1] <li class="nav-item"><a href="#" class="nav-link">Identify</a></li>  
## [2] <li class="nav-item"><a href="#" class="nav-link active">Search</a>  
</li>  
## [3] <li class="mobile--hidden nav-item"><a href="#" class="nav-  
link">Tools</a></li>  
## [4] <li class="nav-item"><a href="#" class="nav-link">Layers</a></li>
```

See the `nav-link active` class on the second element with the "Search" text?

3. Search by Address

Now **enter text** into the desired fields

```
# Find house number field
session %>%
  find_element("div.service-accord") %>%
  elem_children()
```

```
## { selenider_elements (2) }
## [1] <div class="__title"><h3>Search</h3>
<button class="btn btn-info"><i class="fa ...
## [2] <div class="__content"><div
class="service-input form-group"><label
for="input ...
```

3. Search by Address

Now **enter text** into the desired fields

Identify Search Tools Layers

Search

Parcel Number

Owner Name

House Number

Street Name

Close Go

Results will render here.

```
# Find house number field
```

```
session %>%
```

```
  find_element("div.__content") %>%
```

```
  elem_children()
```

```
## { selenider_elements (5) }
```

```
## [1] <div class="service-input form-group">  
<label for="input-b030ec9a-1795-4138-bd ...
```

```
## [2] <div class="service-input form-group">  
<label for="input-026d88bb-d4fb-40b0-96 ...
```

```
## [3] <div class="service-input form-group">  
<label for="input-e48862ea-86db-4da4-9e ...
```

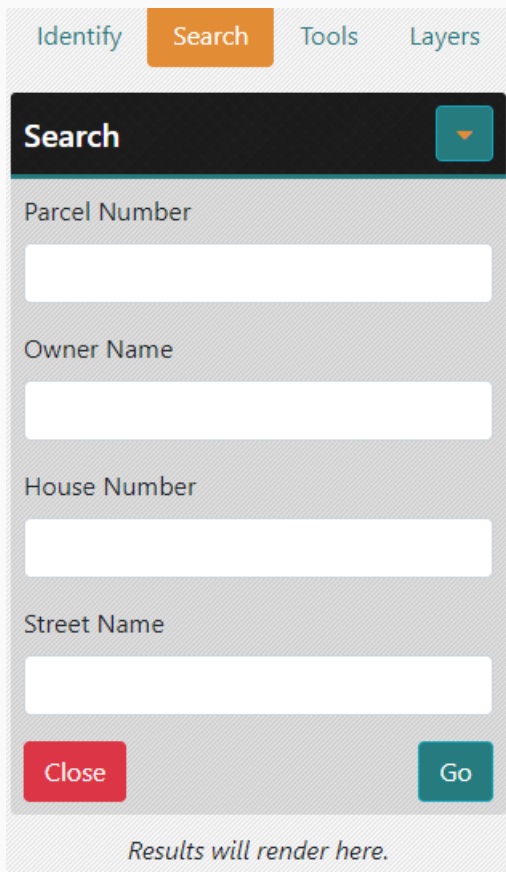
```
## [4] <div class="service-input form-group">  
<label for="input-ae0aed67-6ac6-48d4-ab ...
```

```
## [5] <div class="tab-controls mr-auto">  
<button class="btn btn-danger"><i class="cl
```

```
...
```

3. Search by Address

Now **enter text** into the desired fields and click "Go"



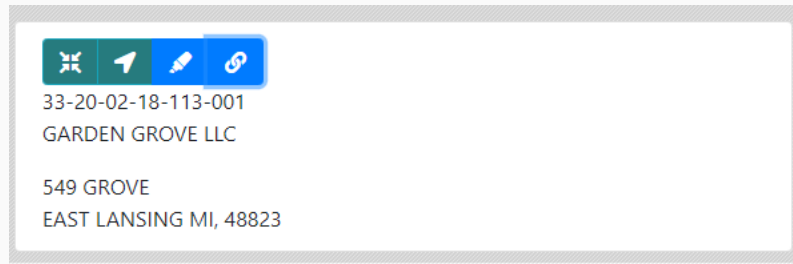
The screenshot shows a search interface with a top navigation bar containing 'Identify', 'Search' (highlighted in orange), 'Tools', and 'Layers'. Below this is a 'Search' panel with a dark header and a dropdown arrow. The panel contains four input fields: 'Parcel Number', 'Owner Name', 'House Number', and 'Street Name'. At the bottom of the panel are two buttons: a red 'Close' button and a teal 'Go' button. Below the panel, a text label reads 'Results will render here.'

```
# set house number
ss("div.service-input.form-group") %>%
  elem_find(has_text("House")) %>%
  find_element("input") %>%
  elem_set_value("549")
```

```
# set street name
ss("div.service-input.form-group") %>%
  elem_find(has_text("Street")) %>%
  find_element("input") %>%
  elem_set_value("Grove")
```

```
# Click button
ss("button.btn.btn-info") %>%
  elem_find(has_text("Go")) %>%
  elem_scroll_to() %>%
  elem_click()
```

4. Retrieve Displayed Info



```
session %>%  
  find_element("div.result-item") %>%  
  read_html() %>%  
  html_text2() %>%  
  str_replace_all("\n\n", "\n") %>%  
  str_split("\n")
```

```
## [[1]]  
## [1] "33-20-02-18-113-001"      "garden grove llc"      "549 grove"  
## [4] "east lansing MI, 48823"
```

5. Follow Detail Link

```
s("a.btn.btn-primary") %>%  
  elem_attr("href") %>%  
  open_url()
```

```
current_url()
```

```
## [1] "https://bsaonline.com/SiteSearch/SiteSearchDetails?  
SearchFocus=Assessing&SearchCategory=Parcel+Number&SearchText=33-20-02-18-  
113-001&uid=383&PageIndex=1&ReferenceKey=33-20-02-18-113-  
001&ReferenceType=0&SortBy=&SearchOrigin=0&RecordKey=1%3d33-20-02-18-113-  
001&RecordKeyType=1%3d0&sitetransition=true"
```

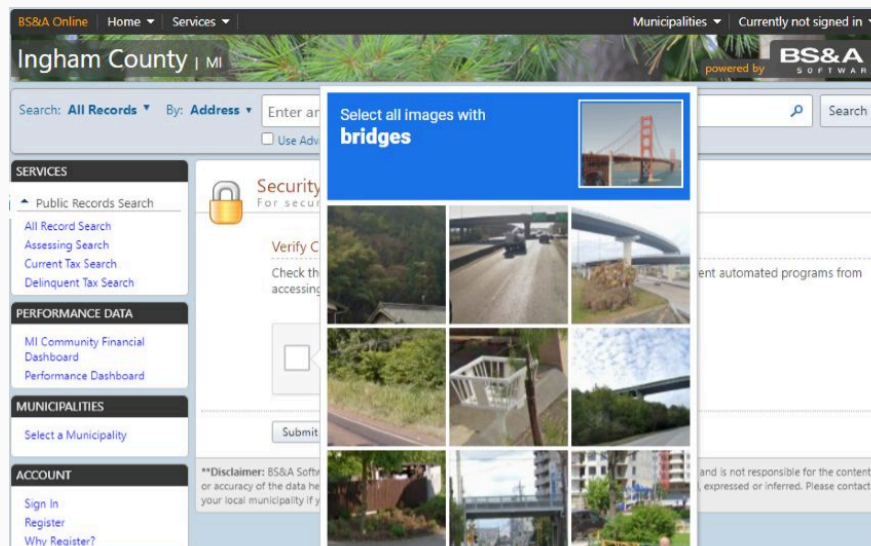
6. Pass Verification Test

Here's where things break down a bit.

We can get our browser to click the "Verify" button

```
s("#GetUserValidationInfo") %>%  
  elem_scroll_to() %>%  
  elem_click()
```

But then we get... the dreaded captcha



6-7. Pass Verification Test + Scrape

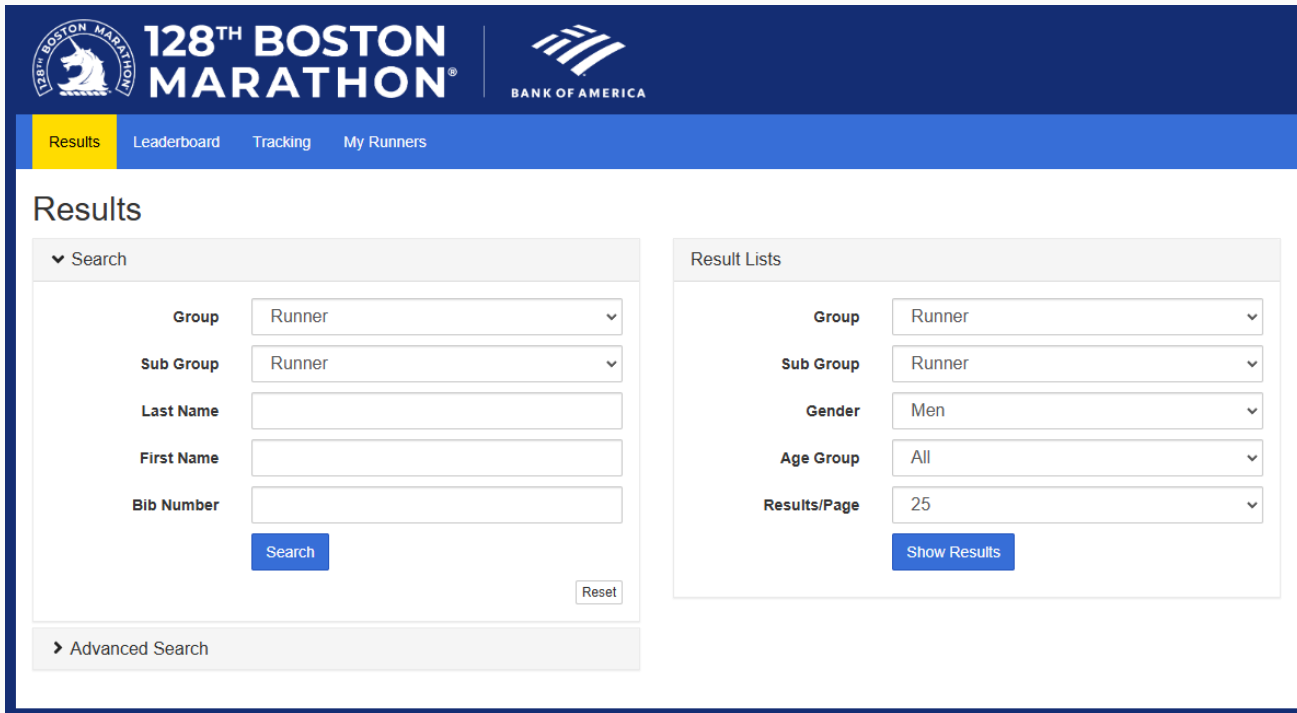
While I have yet to figure out a way around one of these image recognition captchas, here's a **semi-automated workaround**:

- Wait a set amount of time until an HTML element is present with `elem_expect()` and `is_present()`
 - i.e. element from the results page after the captcha
- Manually complete the captcha (must be viewing the session!)
- Proceed to scrape what's on the page

```
details ← ss("div.record-details-collapsible-box") %>%  
  elem_find(has_text("Building Information")) %>%  
  find_element("div.widthContainer table.detailTable") %>%  
  elem_expect(is_present, timeout = 60) %>% # wait up to 60 seconds to  
manually complete the captcha  
  read_html() %>%  
  html_table() %>%  
  as.data.frame()
```

Back to Boston Marathon

Now that we've learned how to use dynamic scraping methods with **selenium**, let's go back to our initial motivation: scraping **2024 Boston Marathon results**



The screenshot shows the official website for the 128th Boston Marathon, sponsored by Bank of America. The page has a dark blue header with the marathon logo and the Bank of America logo. Below the header is a navigation bar with links to Results, Leaderboard, Tracking, and My Runners. The main content area is titled "Results" and contains two search filters. The left filter, labeled "Search", includes dropdowns for Group (Runner) and Sub Group (Runner), and text input fields for Last Name, First Name, and Bib Number. It also has a "Search" button and a "Reset" button. The right filter, labeled "Result Lists", includes dropdowns for Group (Runner), Sub Group (Runner), Gender (Men), Age Group (All), and Results/Page (25). It has a "Show Results" button. At the bottom of the left filter is a link to "Advanced Search".

128TH BOSTON MARATHON[®] | BANK OF AMERICA

Results | Leaderboard | Tracking | My Runners

Results

▼ Search

Group: Runner

Sub Group: Runner

Last Name:

First Name:

Bib Number:

Search

Reset

► Advanced Search

Result Lists

Group: Runner

Sub Group: Runner

Gender: Men

Age Group: All

Results/Page: 25

Show Results

Back to Boston Marathon

Workflow:

- Start a **selenider** session (or use the currently-open one)
- Navigate to the [2024 Boston Marathon results search page](#)
- Select the "Result Lists" form (`form_lists_default`)
- Use the "Age Group" form to select the "18-39" option
- Use the "Results/Page" dropdown to select 100

Back to Boston Marathon

First, navigate to the [2024 Boston Marathon results search page](https://results.baa.org/2024/?pid=start&pidp=start)

```
open_url("https://results.baa.org/2024/?pid=start&pidp=start")
```

And select the form using the `form_lists_default` name:

```
find_element(session, name = "form_lists_default")
```

```
## { selenider_element }  
## <form class="form-horizontal" name="form_lists_default"  
id="form_lists_defaul ...  
## \n<fieldset = "fs-event"="" class="fs-event">\n<input type="hidden"  
name="la ...  
## </form>
```

Back to Boston Marathon

Within the form, we want to interact with a `select` element. To do this, use `elem_select()` on the desired option

- Here, I used XPath syntax for the "18-39" option.
- SelectorGadget gives *another* XPath that works: `//*[@id="default-lists-age_class"]/option[2]`

```
find_element(session, name = "form_lists_default")
%>%
  find_element(name = "search[age_class]") %>% #
  select the "Ag Group" dropdown
  find_element(xpath = "//*/option[@value =
\"1\"]") %>%
  elem_select()
```

```
<select placeholder class="form-control" name="search[age_class]"
id="default-lists-age_class">
  <option value="">All</option>
  <option value="1">18-39</option>
  <option value="2" selected="selected">40-44</option>
  <option value="9">45-49</option>
  <option value="3">50-54</option>
  <option value="10">55-59</option>
  <option value="4">60-64</option>
  <option value="11">65-69</option>
  <option value="8">70-74</option>
  <option value="12">75-79</option>
  <option value="13">80+</option>
</select>
```

Back to Boston Marathon

Next, use the same approach with `elem_select()` to change the results per page option to 100

- This time, using SelectorGadget's CSS selector!

```
find_element(session, name = "form_lists_default")
%>%
  find_element(name = "num_results") %>% # select
  the "Ag Group" dropdown
  find_element("#default-num_results > option:nth-
  child(3)")%>%
  elem_select()
```

```
<select class="form-control" name="num_results" id="default-num_results">
  <option value="25" selected="selected">25</option>
  <option value="50">50</option>
  <option value="100">100</option>
  <option value="500">500</option>
  <option value="1000">1000</option>
</select>
```

Back to Boston Marathon

Finally, find and click the button!

```
find_element(session, name = "form_lists_default") %>%  
  find_element(name = "submit") %>%  
  elem_click()
```

```
▼ <fieldset class="fs-buttons">  
  ▼ <div class="form-group field-submit">  
    ::before  
    ▼ <div class="col-sm-offset-4 col-sm-8"> == $0  
      <button class="btn btn-primary selectorgadget_selected" type="submit"  
        name="submit" id="default-submit">Show Results</button>  
    </div>  
    ::after  
  </div>  
</fieldset>
```

Back to Boston Marathon

Our page has now been populated with the matching search results!

Retrieve the page's current HTML with `get_page_source()`, and then we can use **rvest** methods to retrieve the text we want.

```
Sys.sleep(5)
res_page1 ← get_page_source() %>%
  html_elements("div.row") %>%
  html_elements("li.list-group-item.row") %>%
  html_text2()
head(res_page1)
```

```
## [1] "Place Overall\nPlace Gender\nPlace
Division\nName\nTeam\nTeam\nBIB\nBIB\nHALF\nHALF\nFinish Net\nFinish
Net\nFinish Gun\nFinish Gun"
## [2] "1\n1\n1\nLemma, Sisay\nTeam\n-\nBIB\n2\nHALF\n01:00:19\nFinish
Net\n02:06:17\nFinish Gun\n02:06:17"
## [3] "2\n2\n2\nEsa, Mohamed\nTeam\n-\nBIB\n8\nHALF\n01:02:16\nFinish
Net\n02:06:58\nFinish Gun\n02:06:58"
## [4] "3\n3\n3\nChebet, Evans\nTeam\n-\nBIB\n1\nHALF\n01:02:08\nFinish
```

Back to Boston Marathon

Challenge: format the page 1 results as a dataframe, splitting into all 9 columns

- Back to data wrangling/cleaning!
- *hint:* use `separate_wider_delim()`

Back to Boston Marathon

Challenge: format the page 1 results as a dataframe, splitting into all 9 columns

- Back to data wrangling/cleaning!
- *hint:* use `separate_wider_delim()`

Back to Boston Marathon

A:

```
page1_df <- data.frame(string = res_page1[2:length(res_page1)]) %>%
  separate_wider_delim(cols = string, delim = "\n",
    names = c("place_overall", "place_gender",
"place_division", "name", "team_dup", "team", "bib_dup", "bib",
"half_dup", "half", "finish_net_dup", "finish_net", "finish_gun_dup",
"finish_gun")) %>%
  dplyr::select(-ends_with("dup"))
```

```
head(page1_df)
```

```
## # A tibble: 6 × 9
```

```
##   place_overall place_gender place_division name      team  bib   half
finish_net
```

```
##   <chr>          <chr>          <chr>          <chr>    <chr> <chr> <chr>
<chr>
```

```
## 1 1          1          1          Lemma,... -      2      01:0...
02:06:17
```

```
## 2 2          2          2          Esa, M... -      8      01:0...48 / 50
02:06:58
```

Scraping Dynamic/Interactive Websites

Methods for scraping dynamic or interactive websites opens up a huge range of possibilities.

- Iterative process of
 - Interact with the website and its elements
 - Extract element contents as with static sites
- Interactive workflows are generally more complex, so worth thinking about tradeoff between just brute forcing vs. coding up a scraping routine

Things will also get exponentially easier once we get through our programming lecture, which will allow you to automate repeated scraping tasks

Table of Contents

1. Prologue
2. Scraping Dynamic Websites